

0. STRATEGIE EXAMEN

Sujet probable : QCM + Data Mart (Client/Produit/Vente/Temps/Magasin) + chargement dimensions/faits + requetes decisionnelles.

Ordre de priorite :

- Reconnaitre dimensions / faits / mesures
- Ecrire INSERT INTO ... SELECT
- Ecrire SELECT / JOIN / GROUP BY / HAVING
- Retenir pieges QCM

1. BUSINESS INTELLIGENCE

BI = ensemble de processus/outils pour transformer les donnees en informations utiles a la decision.

Sources : ERP, CRM, fichiers, ventes, clients.

Traitement : extraire, nettoyer, consolider, historiser.

Consommation : reporting, dashboard, KPI, analyse.

Besoins : centraliser, fiabiliser, consolider, KPI a jour, meilleure decision.

KPI : CA, quantite vendue, marge, taux de retard.

BI ne sert PAS a acclereler les transactions ni automatiser tous les metiers.

2. OLTP vs OLAP

Critere	OLTP	OLAP
But	Activite quotid.	Analyser/piloter
Donnees	Actuelles	Historisees
Util.	Employes	Managers/analysts
Req.	Ins/Upd/Del	SELECT/SUM/GROUP
Exemple	Paieement	CA par an

OLTP = transactions temps reel. OLAP = analyse, agregations, historique. OLTP ne stocke PAS l'historique pour l'analyse.

3. POURQUOI UN DATA WAREHOUSE ?

DW = base decisionnelle orientee analyse, separee de la base operationnelle.

Pourquoi pas analyser sur OLTP ? Grosses requetes ralentissent les transactions ; donnees pas toujours nettoyees/historisees ; sources dispersees, formats differents.

4 proprietes Inmon :

- Subject-oriented** : organise par sujets metier (client, produit, vente), pas par application.
- Integrated** : sources multiples unifiees (noms, formats, codes, devises).
- Time-variant** : garde l'historique ; analyse l'evolution dans le temps.
- Nonvolatile** : on charge et consulte ; on ne modifie pas sans cesse comme OLTP.

Single version of truth : un seul chiffre officiel partage.

DW n'est PAS fait pour saisir les ventes quotidiennes ; il sert a les analyser.

4. DATA MART

Data Mart = sous-ensemble specialise d'un DW, dedie a

un domaine (ventes, RH, finance).

Ex. Data Mart commercial : Dim_Produit, Dim_Client, Dim_Temps, Fait_Vente.

DW = global entreprise Data Mart = service/domaine.

5. ETL / ELT / STAGING

E (Extract) : lire depuis ERP, CRM, Excel, BD, fichiers.

T (Transform) : nettoyer, convertir, uniformiser, calculer.

L (Load) : charger dans staging puis DW/Data Mart.

Transformations typiques : doublons, NULL, dates decomposees (jour/mois/annee), devises, sexe M/F, montant = quantite*prix, mapping cles, jointures avec dimensions.

Staging area : zone temporaire brute/intermediaire avant DW.

Talend/SSIS : outils ETL visuels (source, filtre, agregation, mapping, jointure, destination).

ETL ne fait PAS les transactions metiers. ETL ne calcule PAS automatiquement les dimensions.

6. MODELISATION DIMENSIONNELLE

Table de faits

Ce qu'on mesure. Contient mesures + cles etrangeres (FK) vers dimensions.

Ex. : Fait_Vente(id_vente, id_produit, id_client, id_date, id_magasin, quantiteVentes, montantVentes)

Beaucoup de lignes, peu de colonnes.

Table de dimensions

Axe d'analyse descriptif. Ex : Produit(nom, categorie), Client(ville, segment), Temps(jour, mois, annee).

Peu de lignes, beaucoup de colonnes.

Contient : **Surrogate Key (SK)** = cle technique artificielle, independante de la cle metier, utile pour historiser.

Natural Key (NK) = cle originale du systeme source.

Mesures

Nombre a calculer/agreger : quantite, montant, CA, cout, marge.

- Additives** : SUM sur toutes dimensions (quantite, montant, CA).
- Semi-additives** : SUM valide sur certaines dim. seulement (stock, solde bancaire – pas sur le temps).
- Non-additives** : ratio/pourcentage/taux. Recalculer depuis numerateur/denominateur. JAMAIS SUM direct.

Grain

Niveau exact d'une ligne de la table de faits. Ex : une ligne = une vente d'un produit a un client dans un magasin a une date.

Doit etre defini **avant** construction. Determine la precision max des analyses.

Methode de reconnaissance

Champ numerique a analyser = mesure/fait.

Champ descriptif pour filtrer/grouper = dimension.

Date = Dim_Temps.

id_produit dans Fait_Vente n'est PAS une mesure, c'est une FK.

7. ETOILE, FLOCON, CONSTELLATION

Etoile (Star) : table de faits au centre, dimensions autour denormalisees. Simple, peu de jointures, rapide. **Tres probable a l'examen.**

Flocon (Snowflake) : dimensions normalisees en sous-dimensions (Produit → Categorie). Moins redondant, mais plus de jointures, plus complexe. Non recommande juste pour economiser l'espace.

Constellation (Galaxy) : plusieurs tables de faits partageant des dimensions. Ex : Fait_Vente + Fait_Livraison.

	Etoile	Flocon	Gal.
Faits	1	1	Plusieurs
Dim.	Plates	Norm.	Partag.
Jointures	Peu	Bcp	Variable
Vitesse	Rapide	Lent	Variable

Schema etoile type examen :

```
graph TD; Dim_Client -.-> Fact_Vente; Dim_Produit -.-> Fact_Vente; Dim_Temps -.-> Fact_Vente; Fact_Vente -.-> Dim_Magasin;
```

8. AJOUTER DIM vs NOUVELLE TABLE FAITS

Ajouter une dimension si : nouvel axe d'analyse relie au meme processus et au meme grain. Conditions : utile, disponible, qualite ok, relation logique, attributs riches.

Ex. : chaque vente a un magasin → ajouter Dim_Magasin et FK id_magasin dans Fait_Vente.

Nouvelle table de faits si : nouveau processus metier OU grain different OU dimensions totalement differentes.

Ex. : ventes (grain vente) vs livraisons (grain colis) vs stocks (snapshot journalier).

9. ROLAP / MOLAP / HOLAP

ROLAP : donnees dans SGBD relationnel (etoile/flocon), requetes SQL generees. Grande taille, moins rapide sur gros agregats.

MOLAP : cubes multidimensionnels, pre-calculs. Tres rapide pour analyse interactive, limites de taille.

HOLAP : detail dans relationnel + agregats dans cubes. Combine ROLAP et MOLAP.

Tech.	Vitesse	Taille
ROLAP	Lent	Illimitee
MOLAP	Tres rapide	Limitee
HOLAP	Satisfaisant	Grande

MOLAP plus rapide que ROLAP pour les agregats. ERR D QCM.

Operations OLAP : Slice = tranche (1 dim.). Dice = sous-cube (N dim.). Roll-up = resumer (jour→mois→annee). Drill-down = detailler (annee→mois→jour).

10. REGLES D'ASSOCIATION

But : trouver liens/correlations entre produits, comportements d'achat, cross-selling, recommandations.
Ex. : clients qui achètent PC achètent souvent souris.

- **Support** : fréquence de A et B ensemble dans toutes les transactions.
- **Confiance** : parmi ceux qui ont A, proportion qui ont aussi B = P(B|A).
- **Cross-selling** : recommander B si A achète.

Technique	Usage
Regles asso.	Correlations, paniers
Regression	Predire valeur continue
Clustering	Segmenter des groupes
Classification	Predire une categorie

Support + Confiance **UNIQUEMENT** pour regles d'association. Accuracy, CTR, Gain = autres domaines.

11. QCM ANNALE — REPONSES RAPIDES

Q1. Besoins BI

OK A : Centraliser/fiabiliser donnees multi-sources
OK C : Faciliter decisions strategiques
OK E : Obtenir KPI pertinents et a jour

Q2. OLTP / OLAP

OK A : OLTP gere operations temps reel
OK B : OLAP analyse multidimensionnelle
OK D : OLAP optimise calculs/agregations

ERR C : C'est OLAP (pas OLTP) qui stocke l'historique

Q3. ETL

OK A : Extrait sources (ERP, CRM, Excel)
OK C : Transforme, nettoie, normalise
OK D : Charge les donnees dans DWH

ERR B : ETL n'execute PAS les transactions metiers
ERR E : ETL ne calcule PAS les dimensions

Q4. Etapes systeme decisionnel

OK A : Identifier besoins metiers + KPI
OK C : Concevoir modele dimensionnel
OK D : Mettre en place ETL
OK E : Deployer outils visualisation/reporting

ERR B : On ne construit PAS une BD relationnelle transactionnelle pour le decisionnel

Q5. Criteres ajouter une dimension

OK A : Utile pour analyses metier
OK B : Donnees disponibles et de qualite
OK C : Relation logique avec table de faits
OK D : Attributs riches et hierarchiques

Q6. Nouvelle table de faits

OK A : Processus metier different
OK B : Dimensions totalement differentes
OK C : Indicateur pas exprimable au meme grain

Q7. ROLAP / MOLAP / HOLAP

OK A : ROLAP = base relationnelle
OK B : MOLAP = cubes multidim.
OK C : HOLAP = hybride
OK E : MOLAP optimal pour analyses rapides

ERR D : MOLAP PLUS rapide que ROLAP (pas l'inverse)

Q8. Regles d'association

OK A : Correlations entre produits/comportements
OK C : Paniers d'achat frequents
OK D : Liens caches entre evenements

ERR B : Predire variable continue = regression

Q9. Metriques regles d'association

OK A : Support **OK B** : Confiance

ERR C/D/E : Precision, Gain, CTR = autres domaines

Q10. Applications regles d'association

OK A : Recommandation produits (cross-selling)
OK C : Analyse comportements d'achat

ERR B : Prevision ventes = regression. ERR D : Segmentation = clustering

Q11. Talend

OK B : Donnees a jour, consolidees, disponibles
OK D : Transforme toutes varietes de donnees

ERR A : pas TOUS les besoins. ERR C : pas les transactions

Q12. Composantes Talend

OK A : Modification **OK B** : Filtrage
OK C : Agregation **OK D** : Integration

12. DICTIONNAIRE EXPRESS

KPI : indicateur de performance.
Dashboard : tableau de bord.
Reporting : rapports.
Granularite fine : detail. Grossiere : resume.
Ad hoc : requete non prevue a l'avance.
Historisation : garder les anciennes valeurs.
Single version of truth : chiffre officiel partage.
Data-driven decision : decision basee sur donnees.
SCD : Slowly Changing Dimension.

- Type 1 : ecrase l'ancienne valeur.
- Type 2 : ajoute une nouvelle ligne (historique complet).
- Type 3 : ajoute un attribut ancien/nouveau.

13. CREATE TABLE (si demande)

```
CREATE TABLE Dim_Client(  
  id_client INT PRIMARY KEY,  
  nom VARCHAR(50),  
  prenom VARCHAR(50),  
  ville VARCHAR(50),  
  segment VARCHAR(20)  
);  
  
CREATE TABLE Dim_Produit(  
  id_produit INT PRIMARY KEY,  
  nom_produit VARCHAR(100),  
  categorie VARCHAR(50),  
  prix DECIMAL(10,2)  
);  
  
CREATE TABLE Dim_Temps(  
  id_date INT PRIMARY KEY,  
  date_jour DATE,  
  jour INT,  
  mois INT,  
  annee INT  
);  
  
CREATE TABLE Dim_Magasin(  
  id_magasin INT PRIMARY KEY,  
  nom_magasin VARCHAR(100)  
);  
  
CREATE TABLE Fait_Vente(  
  id_vente INT PRIMARY KEY,  
  id_produit INT,  
  id_client INT,  
  id_magasin INT,  
  id_date INT,  
  quantiteVentes INT,  
  montantVentes DECIMAL(12,2),  
  FOREIGN KEY(id_produit)  
    REFERENCES Dim_Produit(id_produit),  
  FOREIGN KEY(id_client)  
    REFERENCES Dim_Client(id_client),  
  FOREIGN KEY(id_magasin)  
    REFERENCES Dim_Magasin(id_magasin),  
  FOREIGN KEY(id_date)  
    REFERENCES Dim_Temps(id_date)  
);
```

14. T-SQL — STRUCTURE & RECAP

Ordre execution : FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

SUM(x)	Somme
AVG(x)	Moyenne
Fonctions d'agregation : COUNT(*)	Nb lignes
COUNT(DISTINCT id)	Nb distincts
MIN(x) / MAX(x)	Min / Max

WHERE vs HAVING

WHERE : filtre les lignes avant agregation.
HAVING : filtre les groupes apres agregation.

```
WHERE dt.annee = 2025  
GROUP BY dp.nom_produit, dm.nom_magasin  
HAVING SUM(f.montantVentes) > 10000
```

JAMAIS WHERE SUM(...)>10000. Correct : HAVING SUM(...)>10000

Sous-requetes

JAMAIS WHERE Price = MAX(Price) — utiliser sous-requete !

```
WHERE s.SalesYTD =  
  (SELECT MAX(SalesYTD) FROM Sales.SalesPerson);  
WHERE s.SalesYTD >  
  (SELECT AVG(SalesYTD) FROM Sales.SalesPerson);
```

Fonctions dates

YEAR(d) / MONTH(d) / DAY(d)	Entiers
DATEPART(YEAR,d)	Entier
DATEDIFF(p,d1,d2)	Difference
DATEADD(p,n,d)	Ajoute n unites
GETDATE()	Date/heure serveur

Fonctions chaines

UPPER(s) / LOWER(s)	Casse
LEFT(s,n) / RIGHT(s,n)	n car.
SUBSTRING(s,d,l)	Extrait
CONCAT(s1,s2)	Concatene
REPLACE(s,old,new)	Remplace
LEN(s)	Longueur

Fonctions math + CASE + COALESCE

ROUND(n,dec)	Arrondi
CEILING(n) / FLOOR(n)	Sup / Inf
CAST(x AS type)	Conversion

```
-- CASE (if/else SQL)  
SELECT CASE CreditRating  
  WHEN 1 THEN 'Superieur'  
  WHEN 2 THEN 'Excellent'  
  ELSE 'Autre'  
END AS Libelle FROM Purchasing.Vendor;
```

```
-- COALESCE : 1er non-NULL  
SELECT COALESCE(psc.Name,  
  CONCAT('Pas de sous-cat: ',p.Name)) AS SC  
FROM Production.Product p  
LEFT JOIN Production.ProductSubcategory psc  
ON p.ProductSubcategoryId  
  = psc.ProductSubcategoryId;
```

DISTINCT / TOP / UNION

```
SELECT DISTINCT [Status] FROM Sales.SOH;  
SELECT TOP 10 * FROM Production.Product;  
SELECT TOP 1 PERCENT p.Name, SUM(OrderQty)  
FROM ... GROUP BY p.Name ORDER BY 2 DESC;  
SELECT City FROM Customer  
INTERSECT  
SELECT City FROM Supplier;  
JOIN = fusion horizontale (colonnes). UNION = fusion verticale (lignes, meme nb colonnes).
```

VUES

```
CREATE VIEW vw_Nom AS  
  SELECT col1, col2 FROM t1 JOIN t2 ON ...;  
-- Utiliser comme une table :  
SELECT * FROM vw_Nom WHERE col = 'val';  
DROP VIEW vw_Nom; -- donnees sources conservees
```

Pieges T-SQL recapitulatif

Erreur	Correct
WHERE P=MAX(P)	Sous-requete
WHERE COUNT(.)>10	HAVING
SELECT Nm, SUM(x)	+GROUP BY Nm
JOIN pour empiler lignes	UNION

15. METHODE REQUETE DECISIONNELLE

9 etapes dans l'ordre :

1. Lire la question entierement
2. Repérer les **mesures** demandees (SUM, AVG, COUNT)
3. Repérer les **axes d'analyse** (par produit, par annee, par magasin...)
4. Choisir la **table de faits** (point de depart)
5. Joindre les **dimensions** necessaires (JOIN)
6. Placer les **filtres** avant agregation (WHERE)
7. Grouper par les **axes** (GROUP BY)
8. Filtrer les **resultats agreg.** (HAVING)
9. Trier si demande (ORDER BY)

Correspondances :

- Quantite totale → SUM(f.quantiteVentes)
- Montant total → SUM(f.montantVentes)
- Moyenne → AVG(...)
- Nombre → COUNT(...)
- Par produit/magasin/annee → SELECT + GROUP BY
- annee = 2025 → WHERE dt.annee = 2025
- Montant total > 10000 → HAVING SUM(...)>10000

16. TEMPLATE SELECT DECISIONNEL

```
SELECT
axe1,          -- ex: dp.nom_produit
axe2,          -- ex: dm.nom_magasin
SUM(f.mesure1) AS total_mesure1,
SUM(f.mesure2) AS total_mesure2
FROM Fait_Vente f
JOIN Dim_Produit dp ON f.id_produit = dp.id_produit
JOIN Dim_Client dc ON f.id_client = dc.id_client
JOIN Dim_Temps dt ON f.id_date = dt.id_date
JOIN Dim_Magasin dm ON f.id_magasin = dm.id_magasin
WHERE condition_avant_agregation
GROUP BY axe1, axe2
HAVING condition_sur_agregat
ORDER BY axe1;
```

Toute colonne non agregee dans SELECT DOIT etre dans GROUP BY. WHERE ne contient JAMAIS SUM/AVG. HAVING contient SUM/AVG.

17. SCHEMA ETOILE TYPE EXAMEN

Modele operationnel (source)

```
Client(id_client, nom, prenom, ville, segment)
Produit(id_produit, nom, categorie, prix)
Ventes(id_vente, id_produit, id_client,
        date_vente, quantite, prix_unitaire)
Magasin(id_magasin, nom_magasin)
```

Modele decisionnel etoile

```
Dim_Produit (id_produit PK, nom_produit,
             categorie, prix)
Dim_Client  (id_client PK, nom, prenom,
             ville, segment)
Dim_Temps   (id_date PK, date_jour,
             jour, mois, annee)
Dim_Magasin (id_magasin PK, nom_magasin)
Fait_Vente  (id_vente PK,
             id_produit FK, id_client FK,
             id_date FK, id_magasin FK,
             quantiteVentes, montantVentes)
```

Mesures : quantiteVentes, montantVentes = quantite * prix_unitaire.

Axes : produit/categorie, client/segment, temps/annee, magasin.

18. CHARGEMENT DIMENSIONS (INSERT)

```
-- Dim_Produit
INSERT INTO Dim_Produit
(id_produit, nom_produit, categorie, prix)
SELECT id_produit, nom, categorie, prix
FROM Produit;
```

```
-- Dim_Client
INSERT INTO Dim_Client
(id_client, nom, prenom, ville, segment)
SELECT id_client, nom, prenom, ville, segment
FROM Client;
```

```
-- Dim_Magasin
INSERT INTO Dim_Magasin
(id_magasin, nom_magasin)
SELECT id_magasin, nom_magasin
FROM Magasin;
```

```
-- Dim_Temps (DISTINCT obligatoire !)
INSERT INTO Dim_Temps
(id_date, date_jour, jour, mois, annee)
SELECT DISTINCT
CONVERT(INT,FORMAT(date_vente,'yyyyMMdd'))
AS id_date,
date_vente AS date_jour,
DAY(date_vente) AS jour,
MONTH(date_vente) AS mois,
YEAR(date_vente) AS annee
FROM Ventes;
```

```
-- Alternative simple (si pas d'id_date numerique)
INSERT INTO Dim_Temps
(date_jour, jour, mois, annee)
SELECT DISTINCT
date_vente,
DAY(date_vente) AS jour,
MONTH(date_vente),
YEAR(date_vente)
FROM Ventes;
```

Toujours DISTINCT pour Dim_Temps si plusieurs ventes le meme jour.

19. CHARGEMENT FAIT_VENTE

Sans magasin

```
INSERT INTO Fait_Vente
(id_vente, id_produit, id_client,
id_date, quantiteVentes, montantVentes)
SELECT
v.id_vente,
dp.id_produit,
dc.id_client,
dt.id_date,
v.quantite,
v.quantite * v.prix_unitaire
FROM Ventes v
JOIN Dim_Produit dp
ON v.id_produit = dp.id_produit
JOIN Dim_Client dc
ON v.id_client = dc.id_client
JOIN Dim_Temps dt
ON v.date_vente = dt.date_jour;
```

Avec magasin (apres ajout Dim_Magasin)

```
INSERT INTO Fait_Vente
(id_vente, id_produit, id_client,
id_magasin, id_date,
quantiteVentes, montantVentes)
SELECT
v.id_vente,
dp.id_produit,
dc.id_client,
dm.id_magasin,
dt.id_date,
v.quantite,
v.quantite * v.prix_unitaire
FROM Ventes v
JOIN Dim_Produit dp
ON v.id_produit = dp.id_produit
JOIN Dim_Client dc
ON v.id_client = dc.id_client
JOIN Dim_Magasin dm
ON v.id_magasin = dm.id_magasin
JOIN Dim_Temps dt
ON v.date_vente = dt.date_jour;
```

20. REQUETES DECISIONNELLES COMPLETES

Par produit, magasin et annee

```
SELECT
dp.nom_produit,
dm.nom_magasin,
dt.annee,
SUM(f.quantiteVentes) AS quantite_totale,
SUM(f.montantVentes) AS montant_total
FROM Fait_Vente f
JOIN Dim_Produit dp ON f.id_produit=dp.id_produit
JOIN Dim_Magasin dm ON f.id_magasin=dm.id_magasin
JOIN Dim_Temps dt ON f.id_date =dt.id_date
GROUP BY dp.nom_produit, dm.nom_magasin, dt.annee
ORDER BY dt.annee, dp.nom_produit, dm.nom_magasin;
```

Annee 2025 par produit et magasin

```
SELECT
dp.nom_produit,
dm.nom_magasin,
SUM(f.quantiteVentes) AS quantite_totale,
SUM(f.montantVentes) AS montant_total
FROM Fait_Vente f
JOIN Dim_Produit dp ON f.id_produit=dp.id_produit
JOIN Dim_Magasin dm ON f.id_magasin=dm.id_magasin
JOIN Dim_Temps dt ON f.id_date =dt.id_date
WHERE dt.annee = 2025
GROUP BY dp.nom_produit, dm.nom_magasin
ORDER BY montant_total DESC;
```

annee=2025 est un filtre avant agregation : WHERE, pas HAVING.

Montant total > 10 000 en 2025

```
SELECT
dp.nom_produit,
dm.nom_magasin,
SUM(f.quantiteVentes) AS quantite_totale,
SUM(f.montantVentes) AS montant_total
FROM Fait_Vente f
JOIN Dim_Produit dp ON f.id_produit=dp.id_produit
JOIN Dim_Magasin dm ON f.id_magasin=dm.id_magasin
JOIN Dim_Temps dt ON f.id_date =dt.id_date
WHERE dt.annee = 2025
GROUP BY dp.nom_produit, dm.nom_magasin
HAVING SUM(f.montantVentes) > 10000;
```

Montant > moyenne generale

```
SELECT
dp.nom_produit,
dm.nom_magasin,
SUM(f.montantVentes) AS montant_total
FROM Fait_Vente f
JOIN Dim_Produit dp ON f.id_produit=dp.id_produit
JOIN Dim_Magasin dm ON f.id_magasin=dm.id_magasin
JOIN Dim_Temps dt ON f.id_date =dt.id_date
WHERE dt.annee = 2025
GROUP BY dp.nom_produit, dm.nom_magasin
HAVING SUM(f.montantVentes) > (
SELECT AVG(f2.montantVentes)
FROM Fait_Vente f2
JOIN Dim_Temps dt2 ON f2.id_date=dt2.id_date
WHERE dt2.annee = 2025
);
```

CA par categorie > 50 000 en 2025

```
SELECT
dp.categorie,
dt.annee,
SUM(f.montantVentes) AS chiffre_affaires
FROM Fait_Vente f
JOIN Dim_Produit dp ON f.id_produit=dp.id_produit
JOIN Dim_Temps dt ON f.id_date =dt.id_date
WHERE dt.annee = 2025
GROUP BY dp.categorie, dt.annee
HAVING SUM(f.montantVentes) > 50000
ORDER BY chiffre_affaires DESC;
```

Raisonnement : mesure = montant ; axes = categorie + annee ; filtre avant = 2025 (WHERE) ; filtre apres = CA > 50000 (HAVING).

21. REQUETES T-SQL AVANCEES (AdventureWorks)

Agreg. par annee avec DatePart

```
SELECT
    DATEPART(YEAR, OrderDate) AS annee,
    SUM(TotalDue) AS total
FROM Sales.SalesOrderHeader
GROUP BY DATEPART(YEAR, OrderDate)
ORDER BY annee;
```

TOP 1 PERCENT meilleur produit

```
SELECT TOP 1 PERCENT
    p.Name,
    SUM(sod.OrderQty) AS Total
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p
    ON sod.ProductID = p.ProductID
GROUP BY p.Name
ORDER BY 2 DESC;
```

Sous-categories avec plus de 10 produits

```
SELECT ps.Name, COUNT(p.ProductID) AS Nb
FROM Production.Product p
JOIN Production.ProductSubcategory ps
    ON ps.ProductSubcategoryID
    = p.ProductSubcategoryID
GROUP BY ps.Name
HAVING COUNT(p.ProductID) > 10;
```

Code utilisateur : 3 lettres nom + 1 prenom

```
SELECT DISTINCT LastName, FirstName,
    UPPER(CONCAT(
        SUBSTRING(LastName,1,3),
        SUBSTRING(FirstName,1,1))) AS Code
FROM Person.Person;
```

DATEDIFF - nb jours depuis commande

```
SELECT OrderDate,
    DATEDIFF(DAY, OrderDate, GETDATE()) AS NbJours
FROM Sales.SalesOrderHeader;
```

CASE - traduire code en libelle

```
SELECT CreditRating,
    CASE CreditRating
        WHEN 1 THEN 'Superieur'
        WHEN 2 THEN 'Excellent'
        WHEN 3 THEN 'Au-dessus moy.'
        WHEN 4 THEN 'Moyenne'
        ELSE 'Sous la moyenne'
    END AS Libelle,
    COUNT(*) AS NbFournisseurs
FROM Purchasing.Vendor
GROUP BY CreditRating;
```

COALESCE - gerer les NULL

```
SELECT p.Name AS Produit,
    COALESCE(psc.Name,
        CONCAT('Pas de SC: ', p.Name)) AS SC
FROM Production.Product p
LEFT JOIN Production.ProductSubcategory psc
    ON p.ProductSubcategoryID
    = psc.ProductSubcategoryID;
```

INTERSECT / UNION / EXCEPT

```
-- Villes dans les deux tables
SELECT City FROM Customer
INTERSECT
SELECT City FROM Supplier;

-- Villes dans Customer seulement
SELECT City FROM Customer
EXCEPT
SELECT City FROM Supplier;

-- Concatener texte + nombre
SELECT 'Total : ';
```

```
+ CAST(COUNT(*) AS NVARCHAR(10))
+ ' commandes en 2014.'
FROM Sales.SalesOrderHeader
WHERE YEAR(OrderDate) = 2014;
```

DISTINCT sur COUNT

```
SELECT COUNT(DISTINCT CustomerID) AS NbClients
FROM Sales.SalesOrderHeader;
```

22. ARCHITECTURE BI COMPLETE

Couche	Contenu
1 – Sources	OLTP, ERP, CRM, fichiers
↓	ETL (Extract, Transform, Load)
2 – DWH	Modele dimensionnel
↓	Data Marts (sous-ens. metier)
3 – Analyse	OLAP, Reporting, Mining
↓	Decideurs et analystes

Cas Continental Airlines : 5e compagnie US, 50M passagers. Probleme : silos de donnees, faillite. Solution : Go-Forward Plan (30M\$, 6 ans). Resultat : N°1 en 2004, ROI >1000%.

23. SYNTHESE SCHEMAS & CONCEPTS CLES

Surrogate Key vs Natural Key

SK : PK artificielle generee par DWH (pas du systeme source). Permet de gerer l'historique.

NK : ID original du systeme source. Peut changer ou etre reutilise.

Dimension degeneratee

Dimension stockee directement dans la table de faits (pas de table separee). Ex : N° de commande, N° de ticket. Utile quand la dimension n'a pas d'attributs descriptifs.

Mesures – Rappel critique

Additive : SUM sur toutes dimensions (quantite, revenu, CA).

Semi-additive : SUM sur certaines dim. seulement (solde, stock).

Non-additive : JAMAIS SUM (taux, ratio, prix unitaire). Recalculer depuis numerateur/denominateur.

Regles d'association – Formules

Support(A∩B) = transactions(A∩B) / nb total transactions.

Confiance(A→B) = transactions(A∪B) / transactions(A).

24. CHECKLIST AVANT DE RENDRE

QCM :

- Lire toutes les propositions – plusieurs vraies possibles.
- OLAP stocke l'historique (pas OLTP).
- MOLAP plus rapide que ROLAP.
- ETL ne fait pas les transactions metiers.
- Regles asso : metriques = Support + Confiance seulement.
- Clustering = segmentation. Regression = prediction valeur.

Schema dimensionnel :

- Faits au centre, dimensions autour.
- Mesures dans la table de faits.
- Attributs descriptifs dans les dimensions.
- FK dans Fait, PK dans Dim.

Requetes decisionnelles :

- Chaque axe dans SELECT doit etre dans GROUP BY.
- WHERE avant GROUP BY ; HAVING apres.
- Pour Dim_Temps : DISTINCT + DAY/MON-

TH/YEAR.

- Pour Fait : partir de Ventes v, puis JOIN dimensions.
- montantVentes = quantite * prix_unitaire.
- Si ajout magasin : Dim_Magasin + id_magasin FK + JOIN.
- Ecrire aliases : f, dp, dc, dt, dm.

Si bloque : ecrire le squelette SELECT / FROM / JOIN / WHERE / GROUP BY puis remplir.

25. ANTI-PIEGES QCM RECAPITULATIF

Affirmation FAUSSE	Ce qui est VRAI
OLTP stocke l'historique	C'est OLAP
ETL execute transactions	Non, jamais
Decisonnel = BD 3NF	Modele dimensionnel
ROLAP plus rapide MO-LAP	C'est l'inverse
WHERE COUNT(.)>10	HAVING COUNT(.)>10
WHERE P = MAX(P)	Sous-requete
SUM(taux) possible	Jamais SUM(taux)
JOIN pour empiler lignes	UNION

26. FORMULES A CASER EN REPONSE

Table de faits : contient les mesures et les cles etrangeres vers les dimensions ; elle represente un evenement metier mesurable.

Dimension : point de vue d'analyse avec attributs descriptifs.

Grain : niveau exact d'une ligne de la table de faits.

Surrogate key : cle technique pour remplacer une cle metier et mieux historiser.

Single version of truth : chiffres fiables et partages par toute l'entreprise.

Data-driven decision : decision basee sur donnees, pas sur intuition.

ETL : Extract (sources) → Transform (nettoyer, standardiser) → Load (charger dans DWH).

Data Mart : sous-ensemble du DWH oriente vers un metier specifique.

27. RECAP ULTRA-FINAL — REGLES D'OR

T-SQL

- Agregat dans WHERE → **sous-requete**
- Filtrer un groupe → **HAVING**
- Colonne SELECT non agreee → **GROUP BY**
- Empiler lignes → **UNION** ; colonnes → **JOIN**
- VIEW = SELECT sauvegardee, s'utilise comme une table
- **COALESCE(a,b)** = a si non-NULL, sinon b

DWH & Modelisation

- Table faits : FK + mesures, beaucoup de lignes
- Table dim : SK + NK + attributs, beaucoup de colonnes
- Etoile = rapide (denorm.) ; Flocon = lent (norm.)
- DWH : LOAD + SELECT seulement (jamais UP-DATE/DELETE)
- Mesure additive : SUM partout ; non-additive : recalculer
- Granularite fine = plus de lignes = plus de detail

QCM

- OLAP ≠ OLTP : l'historique appartient a OLAP
- MOLAP > ROLAP en vitesse ; ROLAP > MOLAP en taille
- Regles asso : Support + Confiance (rien d'autre)
- Regression ≠ association ; Clustering ≠ association
- ETL : Extract + Transform + Load (pas de transactions)
- Talend : OK pour transformation/chargement ; NON pour transactions